

**Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет
«Высшая школа экономики»**

Московский институт электроники и математики им. А.Н.Тихонова

Направление подготовки
«10.03.01 Информационная безопасность»
Образовательная программа **«Информационная безопасность»**

О Т Ч Е Т
о прохождении
учебно-исследовательской практики

Студент Кулешов Д. С.
(Фамилия И.О.)

БИБ233
номер группы

Руководитель практики студента:

Приглашенный преподаватель каф.
информационной безопасности киберфизических
систем департамента электронной инженерии
МИЭМ НИУ ВШЭ

должность и место работы

Джанашиа К.М.

Фамилия И.О.

подпись

Руководитель практики от НИУ ВШЭ:

Зав. каф. информационной безопасности
киберфизических систем департамента
электронной инженерии МИЭМ НИУ ВШЭ

должность и место работы

Евсютин О.О.

Фамилия И.О.

подпись

Практика пройдена с оценкой _____

Дата _____

Москва, 2025

СОДЕРЖАНИЕ

1 Введение	3
2 Описание задач практики	4
3 Исполненное индивидуальное задание	5
3.0 Оборудование.....	5
3.1 Начальная теория о сегментации.	5
3.2 Выбор модели	5
3.3 Используемые функции потерь и метрики	6
3.4 Выбор датасета.....	7
3.5 Написание кода	7
3.5.1 Парсер датасета.....	7
3.5.2 Модель	8
3.5.3 Основная часть кода	12
3.5.4 Визуализация.....	14
3.6 Проблемы, возникшие в процессе выполнения поставленных задач.....	16
3.6.1 Проблема с первым датасетом.	16
3.6.2 Пакет torch.	18
3.6.3 Несовместимость с устаревшим кодом.	18
3.6.4 Проблемы с мультипроцессорностью Python на Windows.....	18
3.6.5 Проблема выхода за границы списка, связанная с суффиксом в масках.	18
3.6.6 Неверная сигнатура функции forward.....	19
3.6.7 Боттлneck.....	19
3.6.8 Проблема с данными, оставшимися на разных устройствах.	19
3.6.9 Проблема с механикой работы SegformerFeatureExtractor.	19
3.6.10 Вторая часть проблемы механики работы SegformerFeatureExtractor.....	20
3.7 Результаты	20
4 Заключение.....	22
5 Список использованных источников.....	23

1 Введение

Целью учебно-исследовательской практики является развитие аналитической и исследовательской компетенций, а также практическое применение теоретических и практических знаний, полученных в ходе лекционных и семинарских занятий в течение предшествующего периода обучения.

Задачи практики:

- Получить теоретические знания по принципу работы свёрточных нейросетей.
- Реализовать сегментацию экрана на фото с помощью свёрточной нейросети.
- Обучить созданную нейросеть и добиться хорошего результата на выбранных метриках.

2 Описание задач практики

Необходимо найти любое решение для сегментации предметов на изображениях и рассмотреть его работу на примерах изображений с горящими экранами, попробовать настроить его параметры так, чтобы выделялся именно экран. У примеров изображений можно менять размеры, формат, или отбирать аналогичные изображения самостоятельно.

3 Исполненное индивидуальное задание

3.0 Оборудование

Работа выполнялась на личном персональном компьютере со следующими характеристиками:

- видеокарта NVIDIA GeForce RTX 4060 8 GB
- процессор Intel Core i7-13650HX
- 16 ГБ оперативной памяти DDR4

3.1 Начальная теория о сегментации.

Есть пять видов сегментации: бинарная сегментация, мультиклассовая сегментация, семантическая сегментация, сегментация объектов и паноптическая сегментация.

Бинарная сегментация выполняет задачу «выбора» одного объекта на картинке, присваивая каждому пикселю класс фона или объекта (0 или 1)

Мультиклассовая сегментация делает то же самое, что и бинарная, но с несколькими классами.

Семантическая сегментация определяет также все объекты на фоне, присваивая конкретный класс к каждому пикселю, а не просто обобщая «фоном».

Сегментация объектов (инстанс – сегментация) – сегментация, которая выделяет несколько объектов, но умеет различать объекты одного класса друг от друга (первая собака, вторая собака).

Паноптическая сегментация – это объединение инстанс-сегментации с семантической сегментацией, модель различает не только все объекты на картинке, включая фон, а также различает несколько объектов одного класса в фоне.

В своей работе я буду использовать бинарную сегментацию, где буду искать в качестве объекта – экран монитора, планшета или телефона.

3.2 Выбор модели

Я решил взять модель SegFormer от компании NVIDIA (чисто символически, у меня соответствующая видеокарта). Среди моделей b0-b5 (Рисунок 1) я выбрал самую лёгкую и простую – “segformer-b0-finetuned-ade-512-512”. Для наших целей она достаточная, т.к. мы делаем самую простую сегментацию.

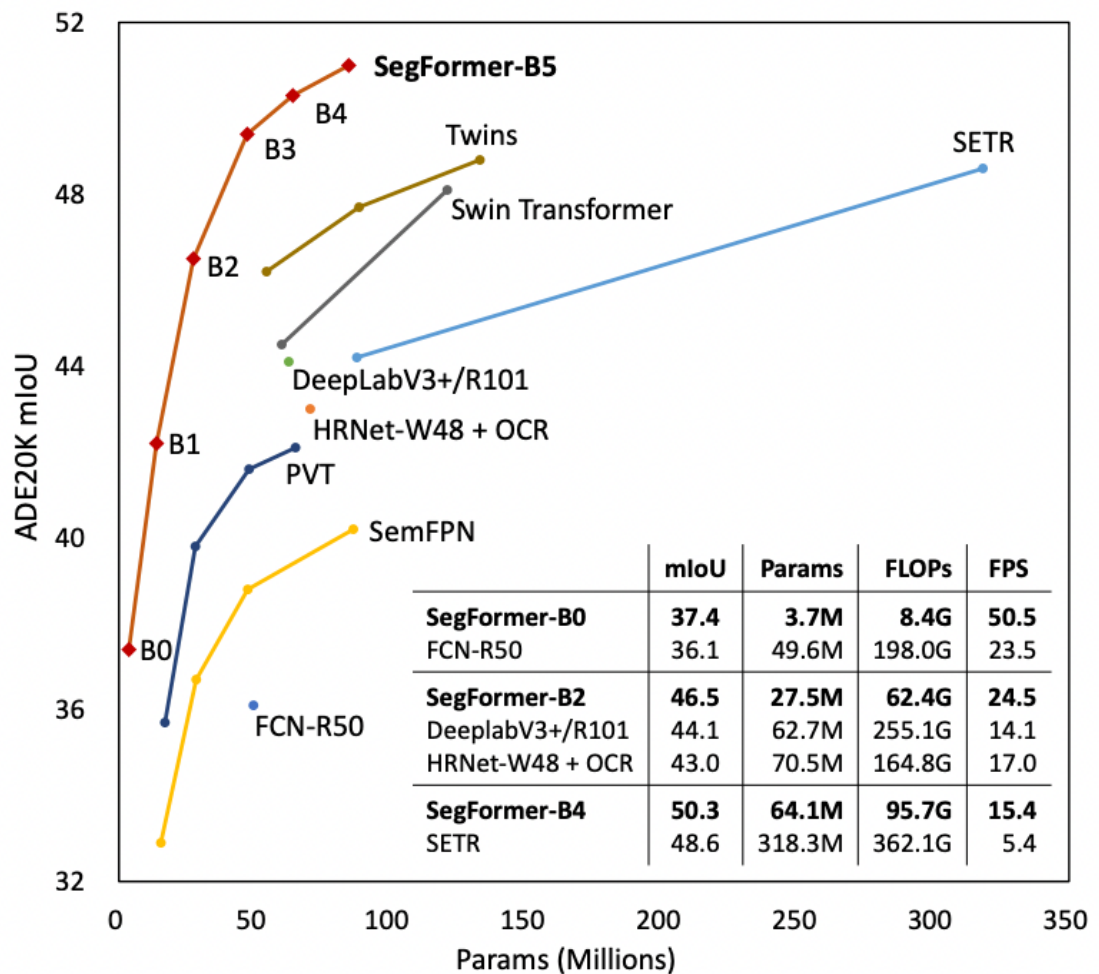


Рисунок 1 - Сравнение различных моделей

3.3 Используемые функции потерь и метрики

Я использовал встроенную в SegFormer функцию потерь под названием log-loss. О ней также рассказывалось на последней прослушанной лекции майнора по машинному обучению (Рисунок 2).

Предсказание вероятностей

$$-\sum_{i=1}^{\ell} \{ [y_i = 1] \log \sigma(\langle w, x_i \rangle) + [y_i = -1] \log(1 - \sigma(\langle w, x_i \rangle)) \} \rightarrow \min_w$$

- Если $y_i = +1$ и $\sigma(\langle w, x_i \rangle) = 0$, то штраф равен $-\log 0 = +\infty$
- Достаточно строго
- Функция потерь называется **log-loss**

$$L(y, z) = -[y = 1] \log z - [y = -1] \log(1 - z)$$

Рисунок 2 - Функция потерь log-loss

Данная функция потерь имеет другое название – Cross Entropy Loss. Данная функция потерь штрафует в зависимости от правильности ответа и уверенности в нём. К примеру, за максимальную уверенность в неправильном ответе, логарифм выдаёт бесконечно отрицательный штраф.

Метрики я использовал следующие: Mean IoU и Mean Accuracy.

Mean IoU - Intersection over Union (Рисунок 3), считает отношение между областью пересечения к области объединения предсказанной и правильной маской.

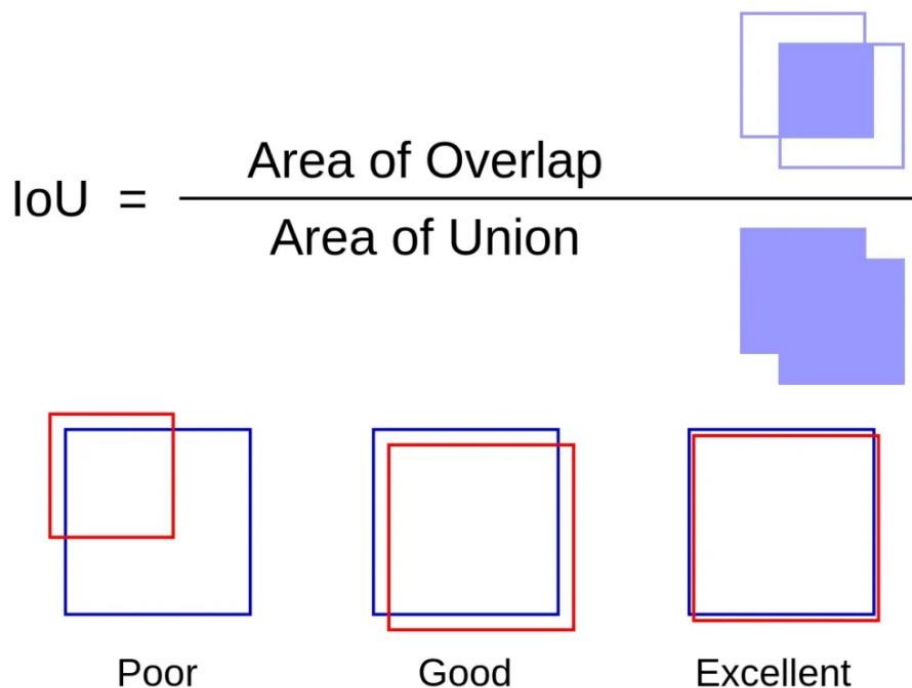


Рисунок 3 - Визуализация IoU

Mean Accuracy – это средняя доля правильно классифицированных пикселей по всем классам.

3.4 Выбор датасета

Изначально я выбрал один датасет^[1], однако в дальнейшем его пришлось заменить на другой^[2]. В этих датасетах нет валидационной выборки, а количество изображений – около 800, отношение train к test – 9 к 1. В первом датасете формат аннотаций – coco.json файл, во втором – csv таблица соответствия значение-класс.

3.5 Написание кода

Весь код представлен в моём github репозитории^[3]. Далее разберём его по частям.

3.5.1 Парсер датасета

Для того, чтобы из папки датасета сделать train и test выборку в виде тензора pytorch, был написан класс SemanticSegmentationDataset (Рисунок 4), который при инициализации принимает путь к датасету и экстрактор и позволяет хранить картинки для дальнейшей работы torch.

```

class SemanticSegmentationDataset(Dataset):
    def __init__(self, root_dir, feature_extractor):
        self.root_dir = root_dir
        self.feature_extractor = feature_extractor

        # Читаем _classes.csv
        self.classes_csv_file = os.path.join(self.root_dir, "_classes.csv")
        with open(self.classes_csv_file, 'r') as fid:
            lines = fid.readlines()
            data = [line.strip().split(',') for line in lines[1:]] # Пропускаем заголовок
            self.id2label = {str(id_): name for id_, name in data}

        # Собираем файлы
        image_file_names = [f for f in os.listdir(self.root_dir) if f.lower().endswith(('.jpg', '.jpeg'))]
        mask_file_names = [f for f in os.listdir(self.root_dir) if f.lower().endswith('.png')]

        # Извлекаем базовые имена, убирая суффикс _mask из масок
        image_bases = {f.replace('.jpg', '').replace('.jpeg', '') for f in image_file_names}
        mask_bases = {f.replace('_mask.png', '').replace('.png', '') for f in mask_file_names}

        # Находим общие базовые имена
        common_bases = sorted(image_bases & mask_bases)
        self.images = [f"{base}.jpg" for base in common_bases] # Предполагаем .jpg, можно добавить .jpeg
        self.masks = [f"{base}_mask.png" for base in common_bases] # Добавляем _mask для масок

        print(f"Images: {len(self.images)}, Masks: {len(self.masks)}")
        print(f"First 5 images: {self.images[:5]}")
        print(f"First 5 masks: {self.masks[:5]}")
        print(f"id2label: {self.id2label}")
        if not self.images or not self.masks:
            raise ValueError(f"Нет парного соответствия изображений и масок в {self.root_dir}")
        if len(self.images) != len(self.masks):
            raise ValueError("Количество изображений и масок не совпадает!")

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        image = Image.open(os.path.join(self.root_dir, self.images[idx]))
        segmentation_map = Image.open(os.path.join(self.root_dir, self.masks[idx]))
        encoded_inputs = self.feature_extractor(image, segmentation_map, return_tensors="pt")
        for k, v in encoded_inputs.items():
            encoded_inputs[k].squeeze_()
        # ОЧЕНЬ ВАЖНАЯ СТРОЧКА
        encoded_inputs['labels'] = (encoded_inputs['labels'] > 0).long() # 0/1/255 -> 0/1

        return encoded_inputs

```

Рисунок 4 - Класс SemanticSegmentationDataset

3.5.2 Модель

Модель представлена в виде класса SegformerFinetuner, который загружает предобученную модель и реализует все необходимые методы для последующего обучения:

- `__init__` - инициализация модели (Рисунок 5). Заполняет все собственные поля класса, в том числе модель (загружается предобученная), метрики, всевозможные выборки, словарь соответствия значений классам и списки для логирования функции потерь.


```

class SegformerFinetuner(pl.LightningModule):

    def __init__(self, id2label, train_data_loader=None, val_data_loader=None, test_data_loader=None, metrics_interval=100):
        super(SegformerFinetuner, self).__init__()
        self.id2label = id2label
        self.metrics_interval = metrics_interval
        self.train_dl = train_data_loader
        self.val_dl = val_data_loader
        self.test_dl = test_data_loader

        self.num_classes = len(id2label.keys())
        self.label2id = {v: k for k, v in self.id2label.items()}

        self.model = SegformerForSemanticSegmentation.from_pretrained(
            "nvidia/segformer-b0-finetuned-ade-512-512",
            return_dict=False,
            num_labels=self.num_classes,
            id2label=self.id2label,
            label2id=self.label2id,
            ignore_mismatched_sizes=True,
        )

        self.train_mean_iou = load("mean_iou")
        self.val_mean_iou = load("mean_iou")
        self.test_mean_iou = load("mean_iou")

        self.validation_step_outputs = []
        self.test_step_outputs = []

```

Рисунок 5 - Метод инициализации класса SegformerFinetuner

- forward – функционал на каждом шаге вычислений (Рисунок 6). Переносит маски и изображения (в виде тензоров) на запрашиваемое устройство (gpu), где происходит вычисление функции потерь, и возвращает значение функции потерь, которое помогает модели понять, как дальше улучшаться.

```

def forward(self, images, masks=None):
    images = images.to(self.device)
    if masks is not None:
        masks = masks.to(self.device)
    # print("\nImages device:", images.device)
    # print("Masks device:", masks.device if masks is not None else "None")
    outputs = self.model(pixel_values=images, labels=masks)
    return outputs

```

Рисунок 6 - Метод forward класса SegformerFinetuner

- training_step – функционал на каждом шаге обучения (Рисунок 7).

```

def training_step(self, batch, batch_nb):
    images, masks = batch['pixel_values'], batch['labels']
    outputs = self(images, masks)
    loss, logits = outputs[0], outputs[1]
    upsampled_logits = nn.functional.interpolate(
        logits,
        size=masks.shape[-2:],
        mode="bilinear",
        align_corners=False
    )
    predicted = upsampled_logits.argmax(dim=1)
    self.train_mean_iou.add_batch(
        predictions=predicted.detach().cpu().numpy(),
        references=masks.detach().cpu().numpy()
    )
    if batch_nb % self.metrics_interval == 0:
        metrics = self.train_mean_iou.compute(
            num_labels=self.num_classes,
            ignore_index=255,
            reduce_labels=False,
        )
        metrics = {'loss': loss, "mean_iou": metrics["mean_iou"], "mean_accuracy": metrics["mean_accuracy"]}
        for k, v in metrics.items():
            self.log(k, v)
        return metrics
    else:
        return {'loss': loss}

```

Рисунок 7 - Метод training_step класса SegformerFinetuner

- validation_step – функционал на каждом шаге валидации (Рисунок 8).

```

def validation_step(self, batch, batch_idx):
    images, masks = batch['pixel_values'], batch['labels']
    outputs = self(images, masks)
    loss, logits = outputs[0], outputs[1]
    upsampled_logits = nn.functional.interpolate(
        logits,
        size=masks.shape[-2:],
        mode="bilinear",
        align_corners=False
    )
    predicted = upsampled_logits.argmax(dim=1)
    self.val_mean_iou.add_batch(
        predictions=predicted.detach().cpu().numpy(),
        references=masks.detach().cpu().numpy()
    )
    self.validation_step_outputs.append({'val_loss': loss})

```

Рисунок 8 - Метод validation_step класса SegformerFinetuner

- on_validation_epoch_end – функционал на конце каждой эпохи валидации (Рисунок 9).

```
def on_validation_epoch_end(self):
    avg_val_loss = torch.stack([x['val_loss'] for x in self.validation_step_outputs]).mean()
    metrics = self.val_mean_iou.compute(
        num_labels=self.num_classes,
        ignore_index=255,
        reduce_labels=False,
    )
    val_mean_iou = metrics["mean_iou"]
    val_mean_accuracy = metrics["mean_accuracy"]
    self.log("val_loss", avg_val_loss, prog_bar=True)
    self.log("val_mean_iou", val_mean_iou, prog_bar=True)
    self.log("val_mean_accuracy", val_mean_accuracy, prog_bar=True)
    self.validation_step_outputs.clear()
```

Рисунок 9 - Метод on_validation_epoch_end класса SegformerFinetuner
- test_step – функционал на конце каждого шага тестирования (Рисунок 10).

```
def test_step(self, batch, batch_idx):
    images, masks = batch['pixel_values'], batch['labels']
    outputs = self(images, masks)
    loss, logits = outputs[0], outputs[1]
    upsampled_logits = nn.functional.interpolate(
        logits,
        size=masks.shape[-2:],
        mode="bilinear",
        align_corners=False
    )
    predicted = upsampled_logits.argmax(dim=1)
    self.test_mean_iou.add_batch(
        predictions=predicted.detach().cpu().numpy(),
        references=masks.detach().cpu().numpy()
    )
    self.test_step_outputs.append({'test_loss': loss})
```

Рисунок 10 - Метод класса test_step класса SegformerFinetuner
- on_test_epoch_end – функционал на конце каждой эпохи тестирования (Рисунок 11).

```
def on_test_epoch_end(self):
    avg_test_loss = torch.stack([x['test_loss'] for x in self.test_step_outputs]).mean()
    metrics = self.test_mean_iou.compute(
        num_labels=self.num_classes,
        ignore_index=255,
        reduce_labels=False,
    )
    test_mean_iou = metrics["mean_iou"]
    test_mean_accuracy = metrics["mean_accuracy"]
    self.log("test_loss", avg_test_loss, prog_bar=True)
    self.log("test_mean_iou", test_mean_iou, prog_bar=True)
    self.log("test_mean_accuracy", test_mean_accuracy, prog_bar=True)
    self.test_step_outputs.clear()
```

Рисунок 11 - Метод on_test_epoch_end класса SegformerFinetuner

- `configure_optimizers` – изменяет параметры для оптимизации обучения, а именно - learning rate в стохастическом градиентном спуске (Рисунок 12).

```
def configure_optimizers(self):  
    return torch.optim.Adam([p for p in self.parameters() if p.requires_grad], lr=2e-05, eps=1e-08)
```

Рисунок 12 - Метод `configure_optimizers` класса `SegformerFinetuner`

- `train_dataloader`, `val_dataloader`, `test_dataloader` – геттеры для выборок (Рисунок 13).

```
def train_dataloader(self):  
    return self.train_dl  
  
def val_dataloader(self):  
    return self.val_dl  
  
def test_dataloader(self):  
    return self.test_dl
```

Рисунок 13 – Методы-геттеры к загрузчикам выборок класса `SegformerFinetuner`

3.5.3 Основная часть кода

После написания всех вспомогательных классов приступим к написанию основной части кода (Рисунок 14). В ней мы должны сначала инициализировать все необходимые классы и функции для обучения и тестирования модели, а именно:

- Скачать датасет, если его ещё нет в папке с проектом (первый, закомментированный блок);
- Инициализировать и настроить экстрактор для датасета;
- Пропарсить весь датасет через наш класс с разделением на train, test и validation выборки;
- Настроить загрузчики датасета для модели (я выбрал размер батча – 32 единицы);
- Инициализировать модель, указав все необходимые параметры;
- Инициализировать менеджер обучения PyTorch, который будет управлять всем процессом обучения (я выбрал максимальное количество эпох – 25 и устройство обучения – gpu).

```

if __name__ == "__main__":
    # Загрузка датасета. Можно не выполнять этот блок кода после первой загрузки файлов
    # rf = Roboflow(api_key="q3YSM0xcnMRqHCb9ppWg")
    # project = rf.workspace("vit-kbay3").project("screen-segmentation")
    # version = project.version(3)
    # dataset = version.download("png-mask-semantic")

    # Настройка экстрактора
    feature_extractor = SegformerFeatureExtractor.from_pretrained("nvidia/segformer-b0-finetuned-ade-512-512")
    feature_extractor.reduce_labels = False
    feature_extractor.size = 128

    # Настройка выборок датасета (т.к. валидационной не предусмотрено, заменяем ее тестовой выборкой)
    train_dataset = SemanticSegmentationDataset("./Screen-segmentation-3/train/", feature_extractor)
    val_dataset = SemanticSegmentationDataset("./Screen-segmentation-3/test/", feature_extractor)
    test_dataset = SemanticSegmentationDataset("./Screen-segmentation-3/test/", feature_extractor)

    # Настройка DataLoader'ов для модели
    batch_size = 32
    train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True, num_workers=0, prefetch_factor=None)
    val_dataloader = DataLoader(val_dataset, batch_size=batch_size, num_workers=0, prefetch_factor=None)
    test_dataloader = DataLoader(test_dataset, batch_size=batch_size, num_workers=0, prefetch_factor=None)

    # Инициализация модели
    segformer_finetuner = SegformerFinetuner(
        train_dataset.id2label,
        train_dataloader=train_dataloader,
        val_dataloader=val_dataloader,
        test_dataloader=test_dataloader,
        metrics_interval=10,
    )

    # Инициализация менеджера обучения из pytorch
    trainer = pl.Trainer(
        accelerator="gpu",
        devices=1,
        precision=16,
        max_epochs=25,
        callbacks=[EarlyStopping(monitor="val_loss", patience=5), ModelCheckpoint(save_top_k=1, monitor="val_loss")]
    )

```

Рисунок 14 - Подготовка всех основных переменных

Далее необходимо запустить обучение, а затем – тестирование, которое покажет успех нашей модели в виде значений выбранных ранее метрик (Рисунок 15).

```

isTrained = 1
if not isTrained:
    trainer.fit(segformer_finetuner)
else:
    segformer_finetuner = SegformerFinetuner.load_from_checkpoint(
        "lightning_logs/version_37/checkpoints/epoch=8-step=459.ckpt",
        id2label=train_dataset.id2label,
        train_dataloader=train_dataloader,
        val_dataloader=val_dataloader,
        test_dataloader=test_dataloader,
        metrics_interval=10,
    )
    trainer.test(segformer_finetuner)

res = {
    "test_loss": trainer.callback_metrics["test_loss"].item(),
    "test_mean_iou": trainer.callback_metrics["test_mean_iou"].item(),
    "test_mean_accuracy": trainer.callback_metrics["test_mean_accuracy"].item()
}
print("Test results:", res)

```

Рисунок 15 - Запуск обучения и тестирования модели, вывод значений метрик

3.5.4 Визуализация

Для отслеживания процесса обучения я использую инструмент tensorboard (Рисунок 16).

```
1 %reload_ext tensorboard
2 %load_ext tensorboard
3 %tensorboard --logdir lightning_logs/
```

Рисунок 16 - Код для запуска tensorboard

Tensorboard читает данные из логов, созданных во время обучения, и строит график зависимости поведения модели от шага. Пример работы на рисунке 17.

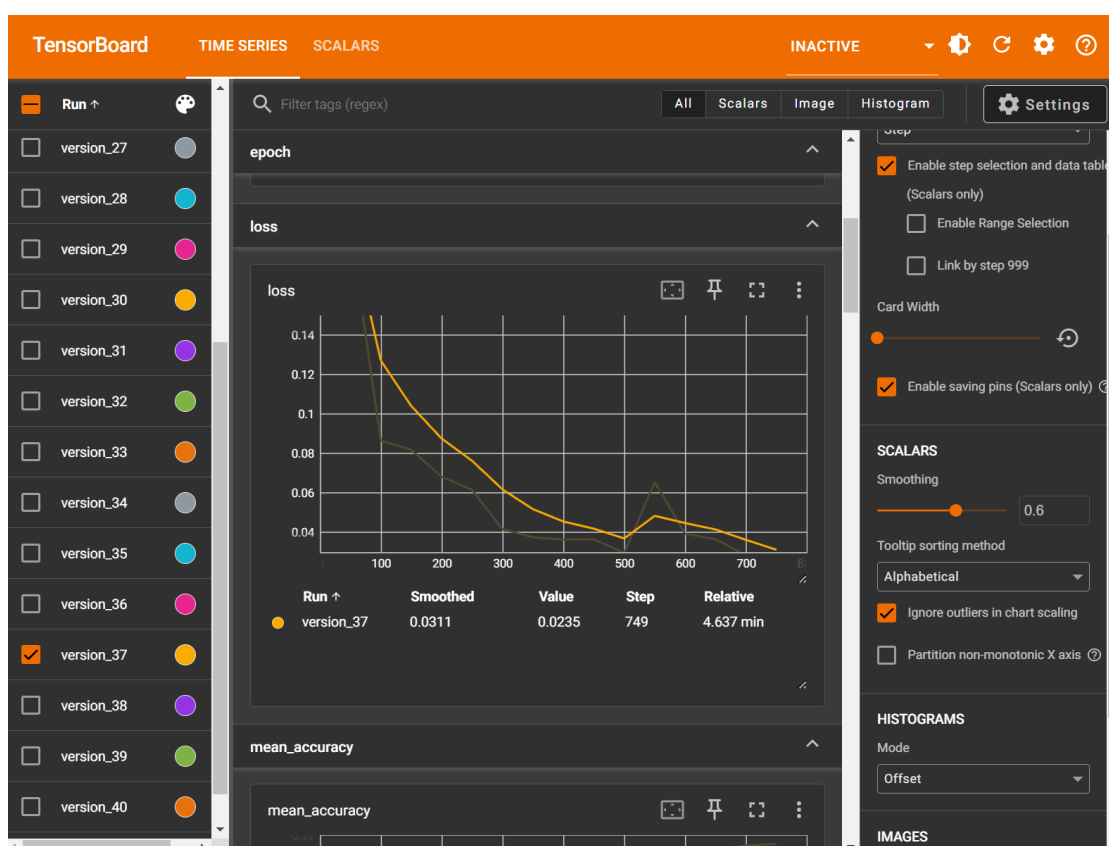


Рисунок 17 - Пример работы tensorboard

Далее, чтобы визуализировать наши результаты и посмотреть, как именно наша нейросеть выделяет экран на изображении, необходимо сделать графики предсказаний нашей модели. Для начала, создадим словарь перевода значения в пиксель (словарь `color_map`), функцию для перевода предсказанного ответа модели в матрицу пикселей (`prediction_to_vis`), а также функцию для получения исходного изображения из тензора (`tensor_to_image`) (Рисунок 18).

```

color_map = {
    0: (0, 0, 0),
    1: (127, 127, 127),
}

def prediction_to_vis(prediction):
    vis_shape = prediction.shape + (3,)
    vis = np.zeros(vis_shape)
    for i, c in color_map.items():
        vis[prediction == i] = color_map[i]
    return Image.fromarray(vis.astype(np.uint8))

def tensor_to_image(tensor):
    """Преобразует тензор PyTorch [3, H, W] в изображение [H, W, 3] для отображения."""
    # Переводим тензор в NumPy, меняем порядок осей (C, H, W) -> (H, W, C)
    image = tensor.cpu().numpy().transpose(1, 2, 0)
    # Денормализуем: feature_extractor нормализует пиксели в диапазон [0, 1] с mean и std,
    # возвращаем их в [0, 255]
    mean = np.array([0.485, 0.456, 0.406]) # Значения из ImageNet
    std = np.array([0.229, 0.224, 0.225])
    image = (image * std + mean) * 255 # Обратная нормализация
    image = np.clip(image, 0, 255).astype(np.uint8)
    return Image.fromarray(image)

```

Рисунок 18 - Вспомогательные функции для визуализации

Сама визуализация написана на matplotlib. Конечная табличка представляет собой три столбца — исходное изображение, маска выделенного экрана, построенная моей моделью и маска выделенного экрана в ответе (Рисунок 19).

```

# Визуализация предсказаний
segformer_finetuner.eval()
segformer_finetuner.to("cuda")
with torch.no_grad():
    for batch in test_dataloader:
        images, masks = batch['pixel_values'], batch['labels']
        images = images.to("cuda")
        masks = masks.to("cuda")
        outputs = segformer_finetuner.model(images, masks)
        loss, logits = outputs[0], outputs[1]
        upsampled_logits = nn.functional.interpolate(
            logits,
            size=masks.shape[-2:],
            mode="bilinear",
            align_corners=False
        )
        predicted = upsampled_logits.argmax(dim=1).cpu().numpy()
        masks = masks.cpu().numpy()
        images = images.cpu() # Переносим изображения обратно на CPU

        print("Predicted unique values:", np.unique(predicted))
        print("Mask unique values:", np.unique(masks))

    f, axarr = plt.subplots(4, 3, figsize=(15, 20))
    for i in range(4):
        # Исходное изображение
        original_image = tensor_to_image(images[i])
        axarr[i, 0].imshow(original_image)
        axarr[i, 0].set_title("Original Image")
        axarr[i, 0].axis("off")

        # Предсказанная маска
        pred_vis = prediction_to_vis(predicted[i, :, :])
        axarr[i, 1].imshow(pred_vis)
        axarr[i, 1].set_title("Predicted")
        axarr[i, 1].axis("off")

        # Истинная маска
        mask_vis = prediction_to_vis(masks[i, :, :])
        axarr[i, 2].imshow(mask_vis)
        axarr[i, 2].set_title("Ground Truth")
        axarr[i, 2].axis("off")
    plt.tight_layout()
    plt.show()
    break

```

Рисунок 19 - Код для построения таблицы визуализации работы программы

3.6 Проблемы, возникшие в процессе выполнения поставленных задач

3.6.1 Проблема с первым датасетом.

Сначала датасет не подходил под мой способ парсинга датасета. Дело в том, что мой первый датасет содержал аннотации в формате .coco.json. В таком формате удобно перечислять несколько объектов сегментации, допустим, если мы по фотографии определяем всех животных в кадре, когда их может быть много. Каждая аннотация содержит класс объекта – category_id, ограничивающий прямоугольник в формате x1, y1, x2, y2 - bbox, площадь объекта сегментации – area, контур полигона объекта сегментации

в формате x1, y1, x2, y2, ..., и флаг, указывающий на скопление – iscrowd (Рисунок 20). Для использования такого файла аннотаций нужно самостоятельно «нарисовать» маску, заполнив каждый пиксель цветом его класса, подобно переводу svg формата в png (векторный формат изображения в растровый). Этот способ удобен для нескольких классов, но для нашей бинарной сегментации, где существует всего 2 класса – фон и объект, это слишком тяжёлый механизм. Мой код ожидал другой формат аннотаций – в виде csv таблицы, где маски уже нарисованы за нас, а файл аннотаций содержит таблицу с двумя столбцами – Pixel Value и Class, в строках – цвет пикселя и класс, которому соответствует данный цвет в маске. Из-за того, что это совершенно разные файлы аннотаций, возникла первая проблема и пришлось полностью поменять датасет.

```
'annotations': [{ 'id': 0,
                  'image_id': 0,
                  'category_id': 1,
                  'bbox': [205, 21, 240, 574.578],
                  'area': 137898.667,
                  'segmentation': [[296,
                                     108.089,
                                     284.606,
                                     148.256,
                                     263.467,
                                     252.207,
                                     258.667,
                                     265.481,
                                     204.8,
                                     491.141,
                                     205.867,
                                     507.259,
                                     211.733,
                                     518.637,
                                     297.6,
                                     591.644,
                                     308.8,
                                     595.437,
                                     322.667,
                                     595.437]],
                  'iscrowd': 0},
  { 'id': 1,
    'image_id': 1,
    'category_id': 1,
    'bbox': [168, 76, 313.5, 538.976]
```

Рисунок 20 - Пример одной аннотации COCO файла

3.6.2 Пакет torch.

Есть разные версии библиотек torch. В том числе есть отдельный пакет torch для взаимодействия с CUDA. Изначально я пытался использовать обычную версию torch для обучения модели, из-за чего моя программа вылетала с ошибкой “MisconfigurationException: No supported gpu backend found!”. Пришлось переустанавливать torch, а также скачивать CUDA, потому что, как оказалось, он не устанавливается с драйверами видеокарты, его нужно загружать отдельно.

3.6.3 Несовместимость с устаревшим кодом.

Из-за обновлений библиотек и устаревшей статьи^[4], которой я воспользовался в ходе выполнения работы, была ошибка: NotImplementedError: Support for `validation\_epoch\_end` has been removed in v2.0.0. `SegformerFinetuner` implements this method. You can use the `on\_validation\_epoch\_end` hook instead. To access outputs, save them in-memory as instance attributes. You can find migration examples in <https://github.com/Lightning-AI/lightning/pull/16520>.

Разобравшись в ней, я понял, что нужно переписать мою функцию validation_epoch_end под свежую версию PytorchLightning - on_validation_epoch_end

3.6.4 Проблемы с мультипроцессорностью Python на Windows.

В попытке обучить модель выдавалась ошибка “RuntimeError: DataLoader worker (pid(s) 20960, 6124, 3364) exited unexpectedly”. Я пытался «распараллелить» вычисления в обучении, поэтому написал num_workers=3 в вызове DataLoader, однако, как оказалось, на Windows мультипроцессорность в DataLoader весьма проблемно настраивается из-за специфики работы Python с процессами, причем эта ошибка имеет массовый характер. Я не переходил на Linux, поэтому решил, что обучение будет происходить без настройки мультипроцессорности. Также я пытался перейти с Jupiter Notebook на обычный .py файл, там мультипроцессорность заработала, но только когда я написал весь основной код (без классов) в блоке “if __name__ == “__main__””. В дальнейшем пришлось перейти обратно на Jupiter Notebook ввиду проблемы под номером 7.

3.6.5 Проблема выхода за границы списка, связанная с суффиксом в масках.

В парсинге картинок возникала ошибка IndexError: list index out of range. Причина данной ошибки – специфика извлечения картинок из датасета. Для того, чтобы понимать, какому файлу соответствует та или иная маска, я должен сначала взять имя файла без .jpg, потом взять файл с тем же именем, но с .png на конце. Я так и сделал, но ошибка не пропала, потому что файлы так и не добавлялись в список масок. Потом я заметил, что файлы с масками перед .png имеют суффикс _mask. После исправления этой недоработки ошибка в парсинге датасета пропала.

3.6.6 Неверная сигнатура функции forward.

После исправлений ошибок с парсингом датасета и его подбором, снова начались ошибки с обучением. На этот раз возникла `NotImplementedError: Module [SegformerFinetuner] is missing the required "forward" function`. Старая версия этой функции содержала параметр `masks`, поэтому сигнатура функции `forward` была неправильной для `pytorch`. Это получилось исправить изменением параметра `masks` в функции - я сделал его параметром по умолчанию `= None`, с этого момента всё заработало.

3.6.7 Боттлнек.

В ходе исследования нагрузки на систему я обнаружил, что нагрузка на процессор увеличилась до предела, что стало причиной явления `bottleneck`, когда процессор не успевает готовить данные для вычисления видеокартой и скорость обучения ограничивалась именно скоростью работы процессора. Из-за этого на графике использования было чётко видно, когда она быстро обрабатывала все приходящие запросы на вычисление и отдыхала примерно то же время в ожидании новой порции заданий от процессора, наблюдалась картина “холмов” с резкими падениями и возрастаниями использования GPU. Не увидив сильных преимуществ `.py` файла над `Jupyter notebook`, я перешёл обратно в `Jupyter` и до самого конца работы программы я установил параметр `num_workers` во всех загрузках датасетов на 0, отказавшись от многопроцессорности, потому что сильного результата это не давало.

3.6.8 Проблема с данными, оставшимися на разных устройствах.

После тестирования модели во время визуализации результатов вылетела следующая ошибка: `RuntimeError: Input type (torch.cuda.FloatTensor) and weight type (torch.FloatTensor) should be the same`. Оказалось, что при загрузке модели из сохранения она может перенестись на `cpu`, поэтому желательно вручную перенести ее на `gpu` (строка `“segformer_finetuner.to("cuda")”`), с помощью которой я делаю дальнейшие действия.

3.6.9 Проблема с механикой работы SegformerFeatureExtractor.

После загрузки картинок для визуализации, я заметил, что там присутствуют пиксели `(0, 0, 0)` и `(255, 255, 255)`, а не `(0, 0, 0)` и `(1, 1, 1)`, как было изначально при просмотре в редакторе. Пришлось спуститься до уровня обработки изображений в экстракторе `SegformerFeatureExtractor`. Оказалось, мои пиксели `(1, 1, 1)` воспринимаются как `grey-scale` и поэтому внутри приводятся к максимальной интенсивности цвета, то есть `(255, 255, 255)`, поэтому во время отрисовки пиксели неправильно отрисовывались в дальнейшем. Это исправилось строкой `“masks = (masks > 0).astype(np.uint8)”` перед вложенным циклом с отрисовкой графиков.

3.6.10 Вторая часть проблемы механики работы SegformerFeatureExtractor.

Теперь видно, что предсказывает модель – это просто черный экран, она не выделяет ни один пиксель экрана. Долго копаясь в коде, спустя два дня я понял, что это происходит из-за неправильных значений пикселей после перевода картинок в тензор. Т.к. пиксели воспринимаются как grey-scale и переводятся в (255, 255, 255) из-за особенности SegformerFeatureExtractor, то их нужно обратно вернуть на место к (1, 1, 1). Это делает строчка “`encoded_inputs['labels'] = (encoded_inputs['labels'] > 0).long() # 0/1/255 -> 0/1`”, которая переводит тензор с масками к пикселям 0/1. Грубо говоря, эта строка меняет нормализацию до шкалы [0; 1], чтобы метки классов на изображении правильно определялись.

3.7 Результаты

После решения всех проблем модель начала работать. Обучение я делал на 32 батчах, $2 \cdot 10^{-5}$ learning rate и 25 эпохах. Динамику разных значений на протяжении всего процесса обучения можно увидеть на графиках в tensorboard на рисунках 21-24.

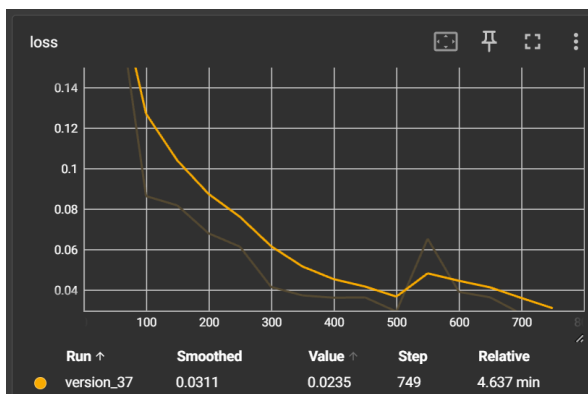


Рисунок 21 - График функции потерь

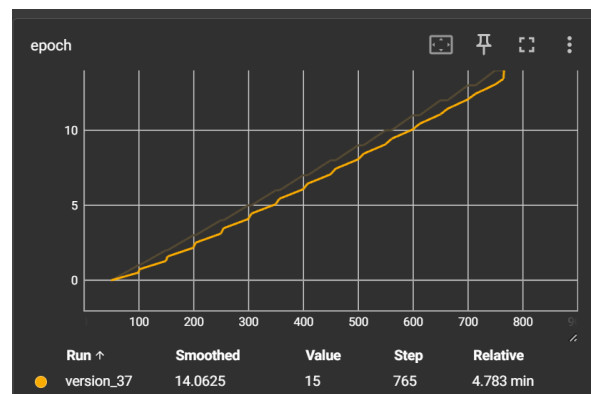


Рисунок 22 - График эпох

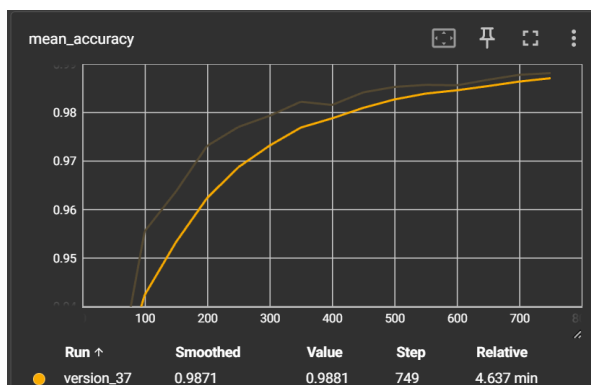


Рисунок 23 - График mean_accuracy

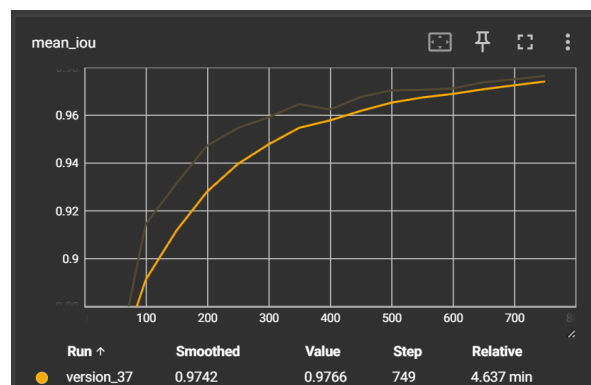


Рисунок 24 - График mean_iou

По окончании тестирования модели получились следующие значения метрик (Рисунок 25):

Test metric	Dataloader 0
test_loss	0.10562332719564438
test_mean_accuracy	0.9647735357284546
test_mean_iou	0.929810106754303

Рисунок 25 – Значения метрик и результат после обучения модели

Визуализация работы модели в выделении экрана на случайных пяти тестах представлена ниже (Рисунок 26).

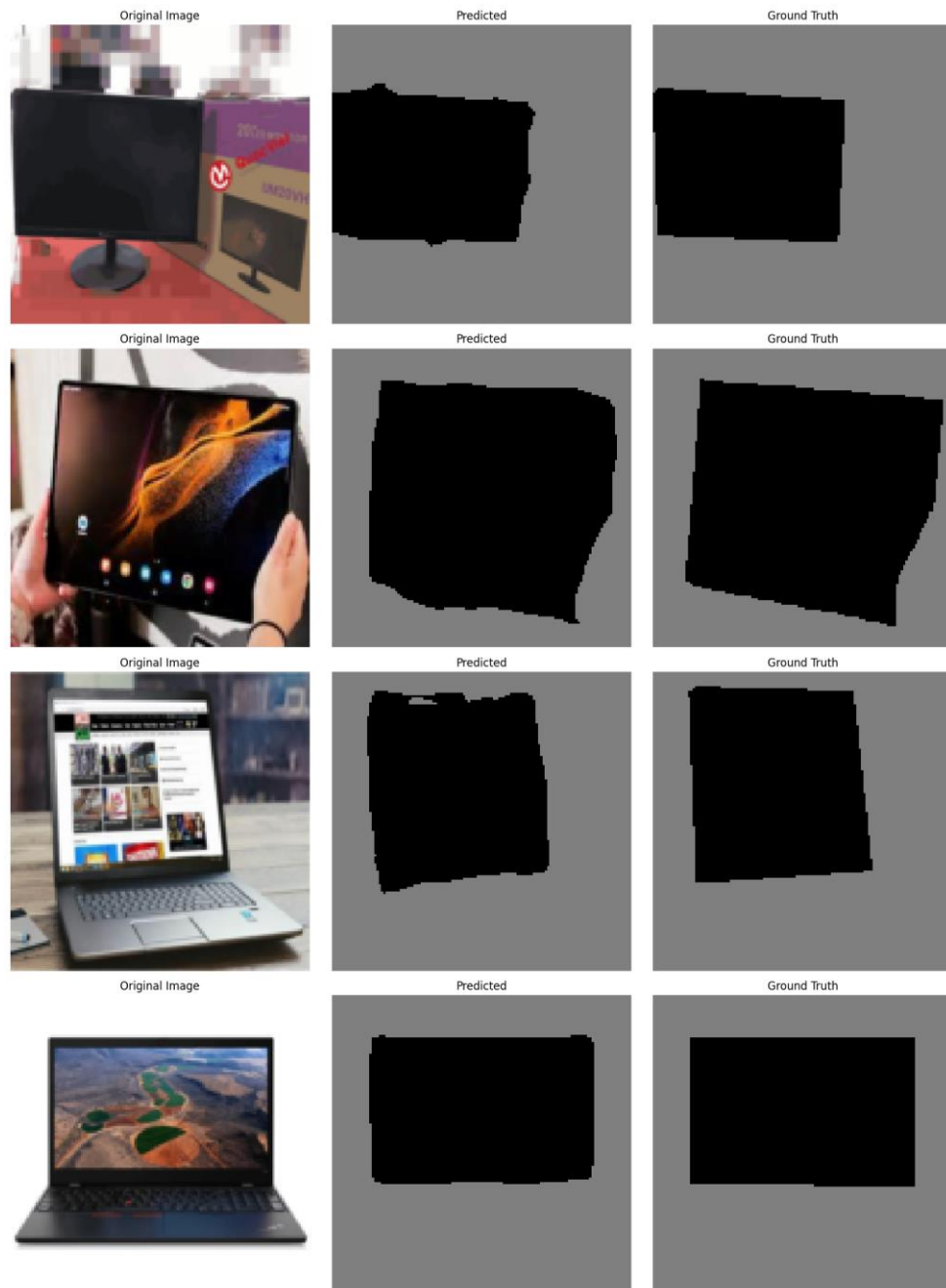


Рисунок 26 - Таблица визуализаций работы программы

4 Заключение

В рамках учебно-исследовательской практики была выполнена работа, целью которой стало развитие аналитических и исследовательских навыков студента. Задача состояла в сегментации экранов на изображениях с использованием современных нейросетевых технологий. Была выбрана бинарная сегментация как вид сегментации, так как больше всего подходит для разбиения пикселей картинки на два класса: экран и фон. В качестве предобученной модели была выбрана Nvidia SegFormer B0, выбор которой продиктован балансом производительности и результативности для нашей задачи.

Для обучения модели использовался специальный датасет, содержащий изображения и соответствующие им маски с выделенным ответом. Датасет представлен в виде изображений формата PNG с CSV-таблицей соответствий значений классам. В процессе разработки использовалась встроенная функция потерь в модель SegFormer - Cross Entropy Loss. Для оценки качества сегментации выбраны следующие метрики: Mean IoU и Mean Accuracy. Подбор гиперпараметров (learning rate, количество эпох, величина одного батча) был выполнен с учётом имеющихся вычислительных ресурсов (один GPU). По итогу были достигнуты следующие результаты: $\text{test_loss} = 0.105$, $\text{test_mean_iou} = 0.93$, $\text{test_mean_accuracy} = 0.965$.

Выполненная работа выполнялась с использованием библиотеки PyTorch. Это значительно упростило управление обучения модели, а также помогло интегрировать модель в инструментарий TensorBoard для визуализации обучения модели. Написанна визуализация результатов, содержащая исходные изображения, предсказанные маски и истинные маски, что позволяет наглядно оценивать качество работы модели. В ходе написания работы было преодолено множество проблем: несоответствие настоящих меток в датасете значениям пикселей на картинках, проблемы с устаревшей логикой работы функций библиотеки и сложность понимания механики работы экстрактора, что приводило к неправильному парсингу датасета. Однако решение вышеперечисленных проблем закрепило полученные умения.

Данная практика помогла усовершенствовать навыки студента в сфере машинного обучения, программирования и анализа данных. Полученные результаты подтверждают успешное решение поставленной задачи.

5 Список использованных источников

1. Screen segmentation Computer Vision Project dataset 2. – URL: <https://universe.roboflow.com/uit-kbay3/screen-segmentation/dataset/2> (дата обращения 15.03.2025).
2. Screen segmentation Computer Vision Project dataset 3. – URL: <https://universe.roboflow.com/uit-kbay3/screen-segmentation/dataset/3> (дата обращения 17.03.2025).
3. DisplaySegmentation by FreeCadelca. – URL: <https://github.com/FreeCadelca/DisplaySegmentation> (дата обращения 15.03.2025).
4. How To Train SegFormer on a Custom Dataset. – URL: <https://blog.roboflow.com/how-to-train-segformer-on-a-custom-dataset-with-pytorch-lightning/> (дата обращения 12.03.2025).